

# Your Workbench, your data

## Lab 1: 40 hands-on minutes on v9

September 2026

Everything you touch today is real **Controlled Tier participant data**. Person-level files never leave the workspace. Printed outputs are aggregate, rounded, and suppression-screened by the scripts — keep them that way. There is no answer key: your numbers are real. This handout gives plausibility checks instead.

### 0–6 minutes: configuration and preflight

```
git clone https://github.com/jeffreyswigert/aou-statgen-lab.git
cd aou-statgen-lab
cp config.example.sh config.sh
nano config.sh          # fill the values projected on screen
bash scripts/00_preflight.sh
```

Fill `WORKSHOP_BUCKET`, `BILLING_PROJECT`, and `CDR_DATASET` from the values the instructor projects. `config.sh` is git-ignored — it never gets committed. Each script explains its own reasoning in comments written for newcomers: read before you run (`less scripts/02_build_phenotype.sh`; `q` exits), starting with `scripts/common.sh`.

### 6–11 minutes: stage genotypes, build the phenotype

```
bash scripts/01_fetch_genotypes.sh
bash scripts/02_build_phenotype.sh
```

While the fetch runs (~1–3 minutes), read the two scripts. Then look at what arrived — two lines each, before trusting anything:

```
head -n 2 lab_data/chr22_filtered.fam    # FID convention?
head -n 2 lab_data/chr22_filtered.bim    # ID scheme?
head -n 2 lab_data/ancestry_preds.tsv    # key column name?
```

**Checkpoints:** samples in the fileset \_\_\_\_\_; variants \_\_\_\_\_.

The phenotype build queries height as program source concept **903133** — not the standard LOINC concept, which would silently pool in EHR heights — for a 1-in-10 teaching subset, then reduces to one row per person (median), applies 100–250 cm bounds, and standardizes within sex. Every one of those clauses is a decision; `scripts/build_pheno.py` states each one.

### 11–16 minutes: phenotype EDA

```
bash scripts/03_explore_phenotype.sh
cat results/aou_pheno_summary.txt
```

The summary is formatted like Stata's `summarize / summarize, detail` — with percentiles where Stata prints Min/Max, because an extreme is one person's exact value. Open `results/aou_pheno_hist.png` and `aou_pheno_kde_sex.png` from the JupyterLab file browser. **Plausibility checks** (not answers): US-cohort sex-specific height means sit near 176 cm (male) and 162 cm (female) with SDs of roughly 7–8 cm; your subset should be in that neighborhood. Funnel counts print rounded to the nearest 100 — explain to your partner why.

### 16–24 minutes: score the PGI

```
bash scripts/04_build_pgi.sh
cat results/aou_pgi_summary.txt
```

**Checkpoints:** weight rows matched (variants processed) \_\_\_\_\_; skipped \_\_\_\_\_. A very low match rate means a wrong ID scheme or wrong column indices — stop and ask. Open `results/aou_pgi_kde_ancestry`. This one-chromosome PGI is deliberately partial. What matters is that you counted every match and every join on real data.

### 24–29 minutes: the regression

```
bash scripts/05_pgi_regression.sh
cat results/aou_pgi_regression.txt
```

The model: standardized height on the standardized PGI, adjusting for age, sex, and five genetic PCs (parsed from AoU’s ancestry file — note its stringified-list column, handled in the script). **Checkpoints:** the PGI coefficient \_\_\_\_\_; its incremental  $R^2$  \_\_\_\_\_. Read the headline row aloud in words: “one SD higher in this PGI is associated with ... SD of height, conditional on the covariates.” Then read the three honest labels at the bottom of the output.

### 29–34 minutes: interpret with a partner

1. Three joins, three key spellings — name them.
2. Why the median per person? Name one alternative and its trade-off.
3. Say the two-sentence honest reading of the ancestry figure (distributions *and* accuracy).
4. Why report the PGI’s *incremental*  $R^2$  rather than its  $R^2$  alone? What would removing the PCs do to the coefficient?
5. What one variable changes to run everyone rather than 1-in-10? All 22 chromosomes?

### 34–40 minutes: save and finish

```
bash scripts/06_save_run.sh
```

In AoU mode the disclosure screen **blocks** the save on small-count findings; today’s tables are designed to pass, and the screen’s report will list your person-level files as workspace-only — read that list. Verify the cloud listing, **stop your cloud app** (VMs bill while idle), and finish the exit check: *I pulled real records, defended every cleaning step, counted every join, and put a real PGI in a model I can defend.*

### GitHub is part of the workflow

You are already using the pattern: this folder is a `git clone`, and `git pull` in it fetches updates onto your VM. Run `git log -oneline | head` to see the audit trail of how these materials evolved, and after your save, `grep code_version results/run_manifest.txt` — every archived run records the exact commit that produced it. The rules that make git safe here: version **code, config templates, and docs**; never commit participant-level files, results, filled configs (`config.sh` is git-ignored on purpose), or notebooks with outputs — a saved `.ipynb` embeds its cell outputs, and one push can carry real data to GitHub. `.gitignore` is a compliance tool: make the safe thing the default in your own repos too.

### Walk the same steps with another variable

The pipeline is a one-variable switch, driven by the menu in `data/phenotypes.tsv` (program measurements and EHR labs, each with vetted concept IDs, bounds, and caveats):

```
bash scripts/02_build_phenotype.sh list
PHENO=ldl bash scripts/02_build_phenotype.sh
bash scripts/03_explore_phenotype.sh
```

DRY\_RUN=1 on step 02 prints the SQL without spending a query. The posted weights are for height, so with another phenotype the step-05 regression is cross-trait — its output will remind you.

### **Taking this to your own project**

Change the concept ID, bounds, and one-row rule for your phenotype; swap in your trait's published weights; set `SAMPLE_MOD=1`; loop `CHROM` over 1–22 and sum the `.sscore SUM` columns. Rehearse on a small `SAMPLE_MOD` and a small chromosome before scaling up — the byte caps and loud failures are there so trial runs stay cheap.